

EE 330 Lab 8 Fall 2020

Digital Synthesis and Automatic Layout

Contents

Objective	3
Checkpoints.....	3
Background	3
Part 1: Verilog Synthesis with RTL Compiler	4
Introduction	4
RTL Compiler.....	4
Setup Standard Cell Library	5
Standard Cell Files for RTL Compiler and Innovus	5
Import Schematic to Cadence Virtuoso after Synthesis.....	8
Validation of Schematic.....	9
Part 2: Layout of Digital Circuits with Innovus	10
Introduction	10
SoC Innovus	10
Running SoC Innovus.....	10
Importing Synthesized Verilog	11
Floorplanning.....	12
Power Planning.....	13
Placement	15
Pin Assignment	15
Routing	16
Filler Cells	17
Verification	17
GDS Export of Layout from Innovus	18
Part 3: Import the layout into Cadence	18
GDS Import of Layout into Virtuoso	19

Possible Errors..... 21

 Red Cells with no Layers 21

DRC and LVS error corrections 21

Notice

Be aware, in this lab, that naming matters. The name of your files *and Verilog modules* will be important as you go through the lab. We designed the lab to have them all be named “boolean_function.” If you name them something different, the commands in this document will not work verbatim. If you use something different, that is fine, but use your name *the same way every place it is used*.

Objective

The purpose of this experiment is to develop methods for using Hardware Description Languages for the design of digital circuits.

Checkpoints

The checkpoints for this lab are as follows:

1. Synthesized Circuit Testbench Results
2. Valid Innovus Connectivity and Geometry Reports
3. Valid DRC and LVS Results

As with all labs, these checkpoints must be shown to a lab TA before the end of your next lab section. You should include these checkpoints in your lab report.

Background

Hardware Description Languages (HDL) are widely used for representing digital systems at both the Behavioral and Structural levels. Full functionality of a digital system can ideally be captured in a HDL and the design of digital circuits is often accomplished within the confines of an HDL framework. At the appropriate level in the HDL, CAD tools can take a design down to the mask level for fabrication in silicon with minimal engineering intervention at the Physical Level in the design process. A system appropriately represented in a HDL can be simulated to predict not only the functionality of sequential and combinational logic circuits, but also timing information about how those circuits are predicted to perform when instantiated in silicon.

The two most widely used HDLs today are Verilog and VHDL. There is considerable similarity between these two languages and engineers are expected to be proficient in both. In this laboratory experiment we will limit our discussion to Verilog which has become more popular in US industry today. Specifically, we will focus on how a HDL can be used for design and simulation of digital integrated circuits.

Appendix A of the Weste and Harris text has a brief discussion of Verilog and students should become familiar with the material in this appendix. There are also numerous books and WEB sites devoted to a discussion of Verilog. Beyond the basic introduction to Verilog discussed in this laboratory experiment, students will be expected to take the initiative to develop their own HDL skills to the level needed to support the digital design component of this course.

Part 1: Verilog Synthesis with RTL Compiler

Introduction

The purpose of this section is to develop the concept of synthesis of digital systems from a hardware language description of the system. This is the intermediate step in a three-step design process which starts with the development of a HDL description (completed in the previous lab), followed by synthesis into a set of standard cells. The third step is placing and routing the cells to create a layout of the circuit which can then be fabricated. The generalized digital circuit design flow, with the topics discussed in this tutorial highlighted, can be found below:

1. Design in HDL (Verilog file)
2. **RTL Compiler (Verilog file → Synthesized Verilog File)**
3. Innovus (Synthesized Verilog file → Layout)
4. Cadence
 - a. Layout Import (Innovus → CIW Import Stream)
 - b. Netlist Import (Synthesized Verilog File → Verilog Import)
5. LVS Verification

RTL Compiler

RTL Compiler is a tool that synthesizes and then optimizes behavioral circuit elements.

Logic synthesis translates textual circuit descriptions like Verilog or VHDL into gate level representations. Optimization minimizes the area of the synthesized design and improves the design's performance. The HDL description can be synthesized into a gate level netlist composed of instances of standard cells.

Once the gate-level netlist is finished, it can be imported into Innovus and used to create a layout. Consistency between the layout and schematic can then be verified with an LVS within Cadence.

Setup Standard Cell Library

First, download the Standard Cell Library – TAR file and RTL Tar File from the course website or Canvas. Then, move them to the ee330 folder for use. They can be extracted through the GUI.

Open Virtuoso. In the CIW, go to Tools → Library Path Editor. This will open a list of libraries and their corresponding paths that Virtuoso sees. In the last row, add the following library and library path:

Library: OSU_stdcells_ami05

Library Path: ~/ee330/OSU_stdcells_ami05

If you click out of the line you just added, the library and library path should change to be blue. This -means the library was successfully added and found. If you entered something wrong, a popup will appear telling you that the directory does not exist and asking if you want to create a new library. If this occurs, hit “Cancel” and check your spelling carefully. Also check that you placed the downloaded files in the ee330 folder before extracting.

Once the library and library path become blue, go to File → Save, and then close the Library Path Editor. In the LMW, go to View → Refresh to refresh your visible libraries. The OSU05 standard cells should now appear. Click around to the different cells to make sure the layout and schematic views can be viewed. The standard cell library should now be available to use.

Standard Cell Files for RTL Compiler and Innovus

Now, the necessary files for use in Genus and Innovus must be obtained to run the digital design flow. We will be using the OSU05 files in this lab. This cell library uses the same AMI 06 process that we should be familiar with from the earlier labs.

From the course website or Canvas, download the rtl.tar.gz file which contains a folder called 'lab8_ee330'. This folder contains the LIB, LEF, and Verilog files for the OSU standard cells. Move rtl.tar.gz into your ee330 folder and extract it.

Now the files are downloaded and extracted, which can be confirmed by looking for the *lab8_ee330* folder in the location used above. Throughout this lab, carefully watch for path names and make sure they are pointing to the correct directories and files.

Now, copy over the Verilog file (boolean_function.v) which was created and tested in Lab 7. The file should be located in your *ee330/verilog* folder if you used ModelSim. Copy the file to *lab8_ee330/rtl/* either using the GUI or the following set of commands:

```
cp ~/ee330/verilog/boolean_function.v ~/ee330/lab8_ee330/rtl/
```

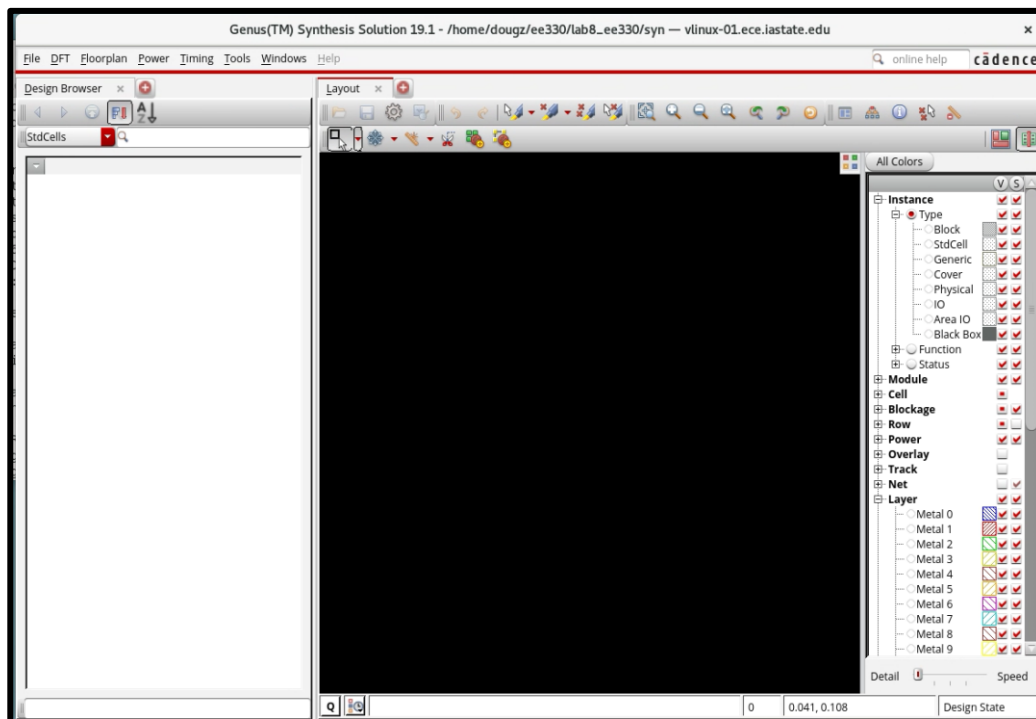
Change directories to the *lab8_ee330/syn/* directory and enter the following commands to begin the RTL Compiler:

```
cd ~/ee330/lab8_ee330/syn
```

```
genus -legacy_ui
```

```
gui_show
```

A GUI will show and your terminal will change to the Genus command line.



In the command window that has a “genus:/>” prompt, type the following to load the correct standard cell libraries and run the synthesis script:

```
set OUTPUT_DIR ./run_dir
```

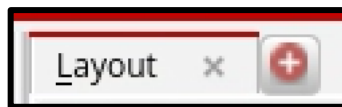
```
set_attribute lib_search_path ../libdir
```

```
set_attribute library {osu05_stdcells.lib}
```

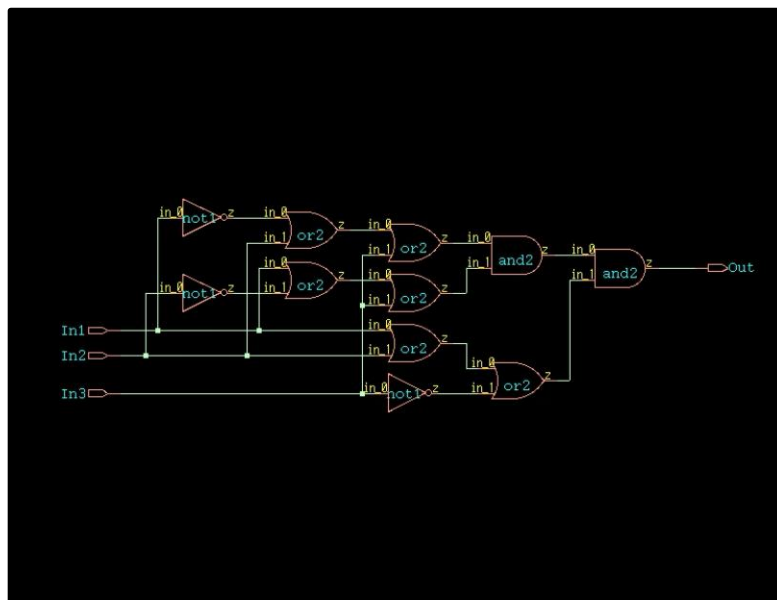
You will get multiple warnings, but no errors. Once the library is set, you need to load and elaborate your Verilog files. In this case, it was placed in the rtl folder and can be processed with the following commands:

```
load -v2001 ../rtl/boolean_function.v
set DESIGN "boolean_function"
elaborate
```

When Verilog is elaborated, each line is evaluated and replaced with a combination of logic gates. Go to your GUI window, there should be an empty “Layout” view. Add a “Schematic” view by clicking on the “Plus” button next to the layout view.



You should get a window with circuit blocks that represent the input Verilog. It should look something like this:



At this point, RTL Compiler has generated an unoptimized version of the Verilog you provided. The final step is “synthesizing” the elaborated design, which optimizes the schematic already created and integrates your design with the AMI standard cell library. While there are many optimization options available to you, we will skip them for this tutorial. The design can be synthesized with the following command:

```
synthesize -to_mapped -eff high -no_incr
```

The GUI window will not automatically update to show the optimized design. To refresh the GUI you need to select your design from the left menu (it will be named “Hier Cell”), then right click -> Schematic View -> In Main. Exporting the synthesized Verilog file is done with these two commands:

```
write -mapped > ${DESIGN}_synth.v
```

```
write_sdc > ${DESIGN}.sdc
```

Congratulations! You have now synthesized your Verilog into a netlist which correspond to a standard cell library! You may now close the Genus GUI. Now, we need to load the synthesized code into Virtuoso for testing and verification that it works as intended.

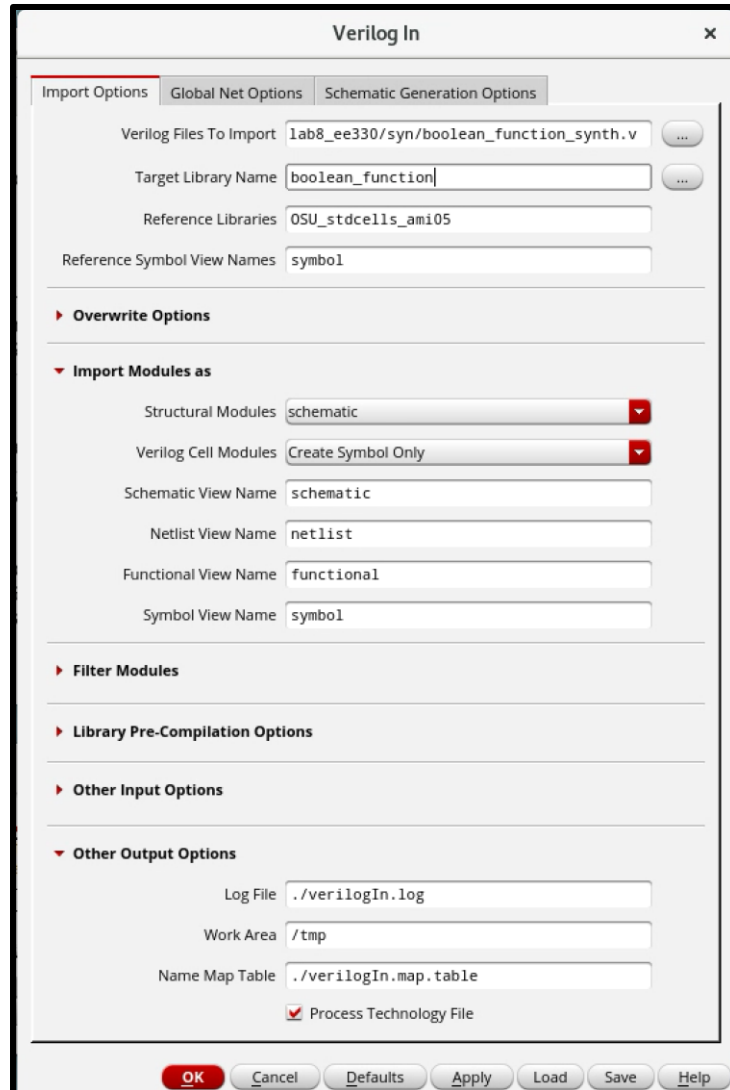
Import Schematic to Cadence Virtuoso after Synthesis

After synthesis, we want to import the gate-level netlist created in RTL Compiler into Cadence Virtuoso. Open Virtuoso and in the library manager create a new library with the name `boolean_function` (or whatever name you want to use for the library for this lab). When the technology file options pop up select “Attach existing technology library” and then select `NCSU_TechLib_ami06`.

Once the library is created, change to the CIW (where the errors and other messages appear) select `File → Import → Verilog`.

The following screen will appear. Fill it in as is shown in the screen shot below, except replace the Verilog file with your synthesized Verilog file created by Genus and use your own library name. This file will end with `_synth.v` if following the above steps.

Make sure all text fields match, with only differences existing if you have named your files differently.



The import process should complete and may have a few errors or warnings. Ignore for now. Go to the schematic cell view created by the import process and open in Virtuoso. The synthesized optimized design should appear with the lowest level of hierarchy being the standard digital blocks (gates) made of transistors in the AMI06 process. If not, something is wrong. **Retry the import process or possibly synthesis before asking for help from the TAs.**

Validation of Schematic

A symbol for your boolean_function should have been created. Create a testbench for the import boolean_function to double check that the generated netlist is functionally equivalent to the Verilog that you wrote and simulated in the previous lab. For V_{DD} , use the “vdd” component discussed in

previous labs. A V_{SS} component is not necessary (V_{SS} is, by default, shorted to GND by Genus). Submit your testbench results as a checkpoint.

Part 2: Layout of Digital Circuits with Innovus

Introduction

In a typical digital design flow, a hardware description language is used to model a design and verify the desired behavior. Once the desired functionality is verified, the HDL description is then taken to a synthesis tool such as RTL compiler. The output of RTL compiler is a gate-level description of the desired circuit. The next step is to take this gate-level description to a place-and-route tool that can convert it to a layout level representation. In this section of the lab, you will be introduced to a place-and-route tool called SoC Innovus. For the purposes of this tutorial, it will be assumed that you have a synthesized Verilog file ready to be placed and routed.

The generalized digital circuit design flow, with the topics discussed in this tutorial highlighted, can be found below:

1. Design in HDL (Verilog file)
2. RTL Compiler (Verilog file → Synthesized Verilog File)
3. **Innovus (Synthesized Verilog file → Layout)**
4. Cadence
 - a. Layout Import (Innovus → CIW Import Stream)
 - b. Netlist Import (Synthesized Verilog File → Verilog Import)
5. LVS Verification

SoC Innovus

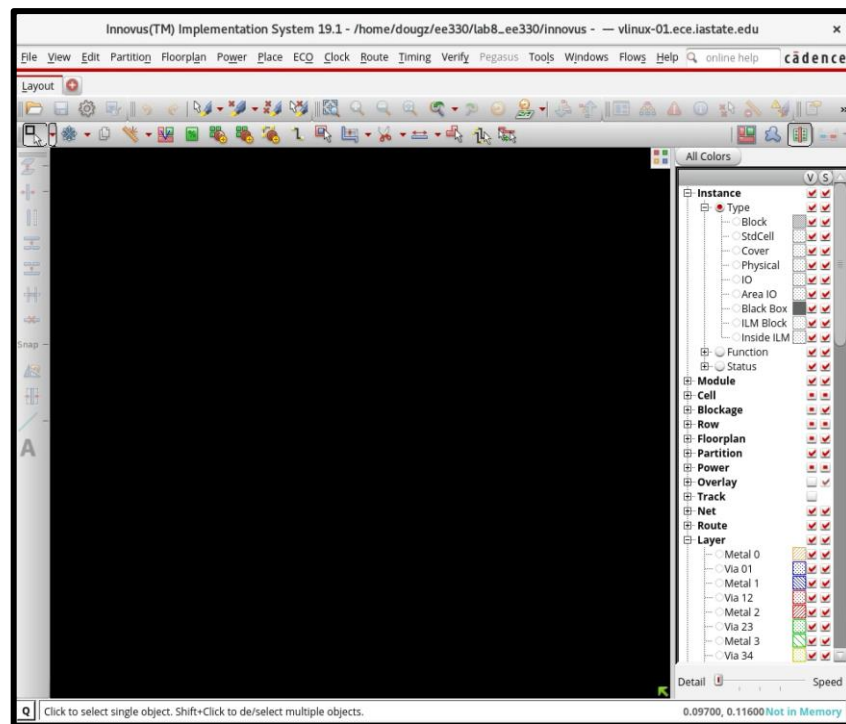
SoC Innovus, in its simplest form, allows a user to create a layout from a synthesized HDL file. Once the synthesized file is imported, Innovus provides a variety of tools to create a floorplan for your design, place standard cells, and automatically connect power rails and local routing. While this lab only provides a brief overview of Innovus, there are many other optimization and layout options available. If you are interested in learning more about Innovus, you may be interested in pursuing CPR E 465.

Running SoC Innovus

In the *lab8_ee330* folder, there should be a folder named Innovus. Enter that folder. Innovus can be started from here with the following command:

```
innovus
```

The command prompt will change to `innovus>` and you will be presented with a GUI, which has been shown below.

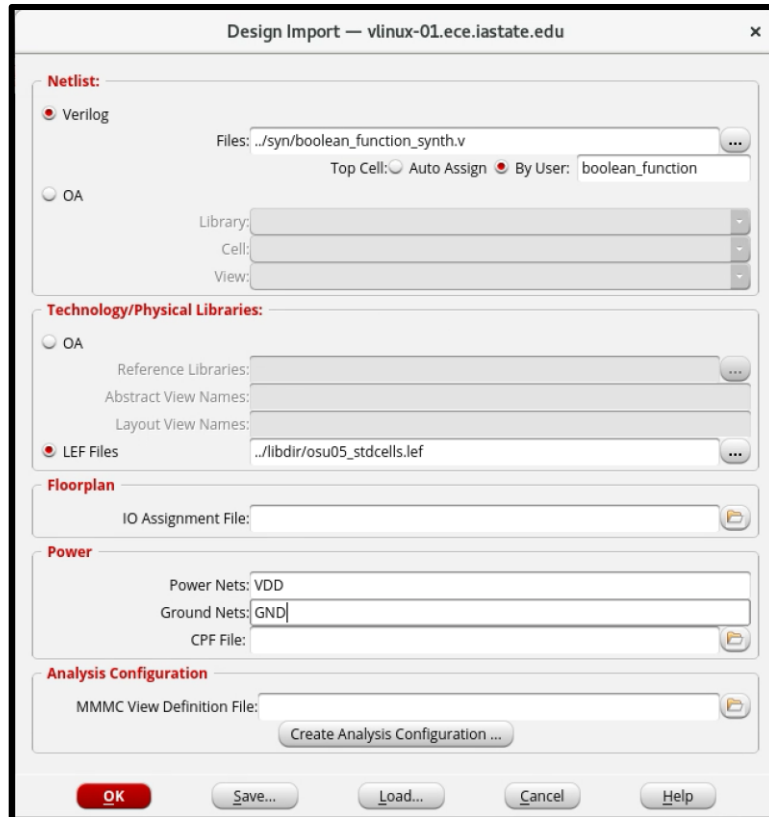


Importing Synthesized Verilog

Once Innovus is initialized, you need to import your synthesized HDL description file. Go to File → Import Design and under Files, select `../syn/boolean_function_synth.v`

Do not import the OA under Technology/Physical Libraries, instead click the LEF Files button and choose `../libdir/osu05_stdcells.lef`

Under Power Nets put VDD and for Ground Nets put GND



Floorplanning

The Innovus GUI should now display an empty “die” and the information box in the bottom-right corner should read “Design is: In Memory.” The next step is to specify a floorplan for your layout. Floorplanning is done to specify the dimensions of your layout and spacing between the core (or area where the standard cells are placed) and power/signal routing. Open up the floorplanning options by clicking on the *Floorplan* menu and selecting *Specify Floorplan*.

Options that you will need to pay attention to are:

- Aspect ratio – height/width ratio of the core.
 - Suggested: 1.0 (this will change to the closest workable number)
- Core Utilization – the amount of core area used for placing standard cells. A larger value means that your design will be fairly compact but routing may be difficult, if not impossible. Smaller values will drastically increase the design area.
 - Suggested: 0.5 (this will change to the closest workable number)
- Core Margins by – specifies the distance between the core and edge of the die. The margin area needed is proportional to the number of input/output (I/O) pins.
 - Suggested: 20 for core to left, right, top, and bottom

Once these values are entered, the layout die/core spacing and aspect ratio should be updated. It may look similar to the image below.



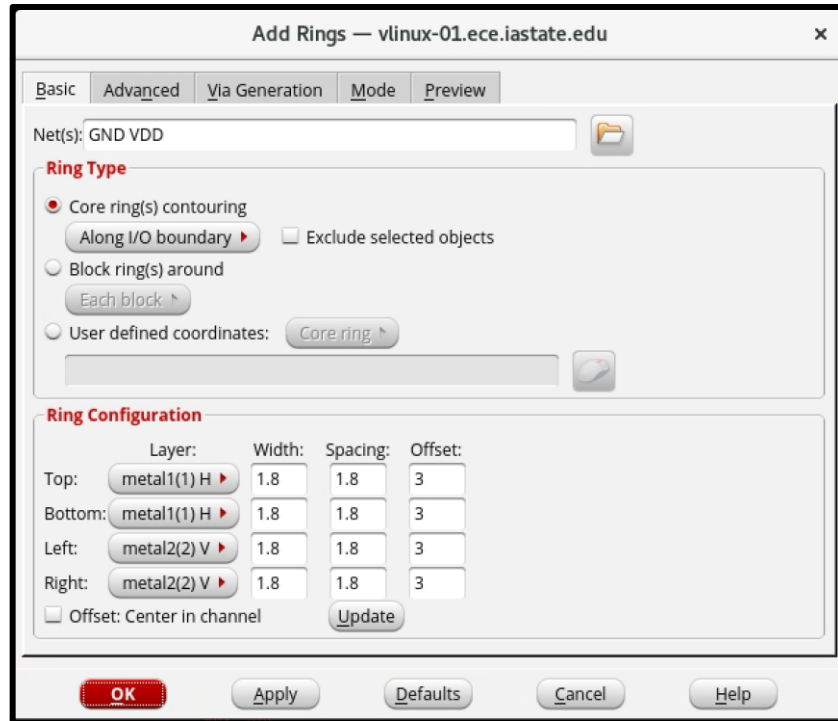
Power Planning

In order to connect your supply voltages to the standard cells, power and ground need to be available on all sides of the die. This is done by adding power rings and specifying global net connections.

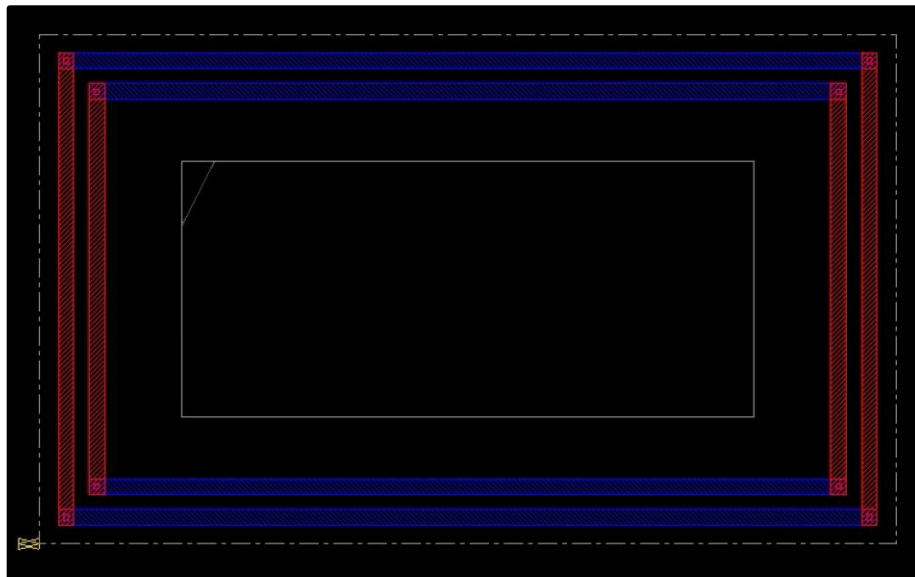
Select the *Power* menu and click on *Power Planning* → *Add Rings*.

A dialog box will pop up and allow you to specify the ring type and configuration. You can plan power in various modes to facilitate various design styles. Since it is preferable to have the routing done inside the power rings, it is a better idea to have them around the I/O boundary so choose this option. If it is not already entered, add “GND VDD” (without quotes) to the Net(s) field.

In the Ring Configuration section, replace the “Offset” for each side with “3”. Leave everything else as “1.8”



Once finished, you should see two rings that run inside the I/O boundary. It should look like the following:



Connection to the global nets is done automatically if the nets are given the same names as was done in the setting of the power nets above. If all was done correctly, power planning should be finished.

Placement

Now we are ready to perform the placement. At the end the cells used in the design will be placed so that they don't overlap and there is enough space for routing in between them. Select the *Place* menu and then *Place Standard Cells*. In the dialog box that appears, leave the default settings and hit OK. In the main GUI window, you can verify that the standard cells were placed by selecting the Physical View Icon in the upper-right:

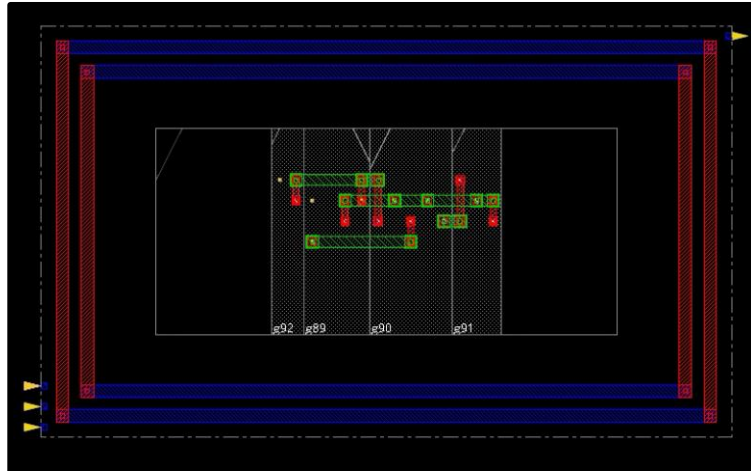


You should see the cells placed in the middle of the core. The IO pins have shifted position accordingly as well.

If placement fails, it is most likely because Innovus cannot place the cells in the given area without causing gate overlaps. To solve this problem, you must start over by reinitializing the floorplan with relaxed constraints. The best constraint to change is the row utilization factor. By lowering this factor, you can increase the likelihood of placement without overlaps.

Pin Assignment

The next step is to assign pin locations for your IO pins. To do that go to *Edit* → *Pin Editor*. This opens up a window which allows you to choose exactly how your pin is setup (which metal layers, location on die boundary). For our purposes we will pick either metal layers 1 or 2, and place the pins anywhere across the boundary (specifying top, left, right, or bottom is more than enough). Select the pin from the menu on the left, then choose “Assign location” under Location tab and pick a location. We will need to do that for all 4 pins. If that is done correctly, arrows indication pin direction should show up on the die's boundary.

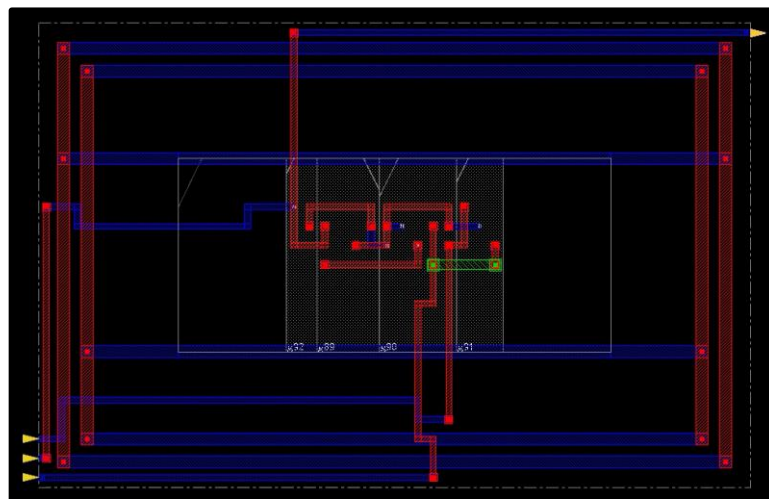


Routing

The built-in routing options within Innovus can now be used to connect the supply nets and create interconnects between the standard cells.

Power and ground routing is performed with the “Special Route” tool. This can be found by selecting the *Route* menu and then *Special Route*. In the dialog box that appears, unselect *Block pins*, *Pad pins*, and *Pad rings*. After clicking OK, you should see horizontal and/or vertical metal power routing added to your design.

Once power routing is complete, the remaining connections between standard cells and the die I/O can be routed using the “NanoRoute” tool. Select *Route* and then *NanoRoute* and finally *Route...* to enter the dialog box. You do not have to make any changes and can click OK. Your design should now resemble the following

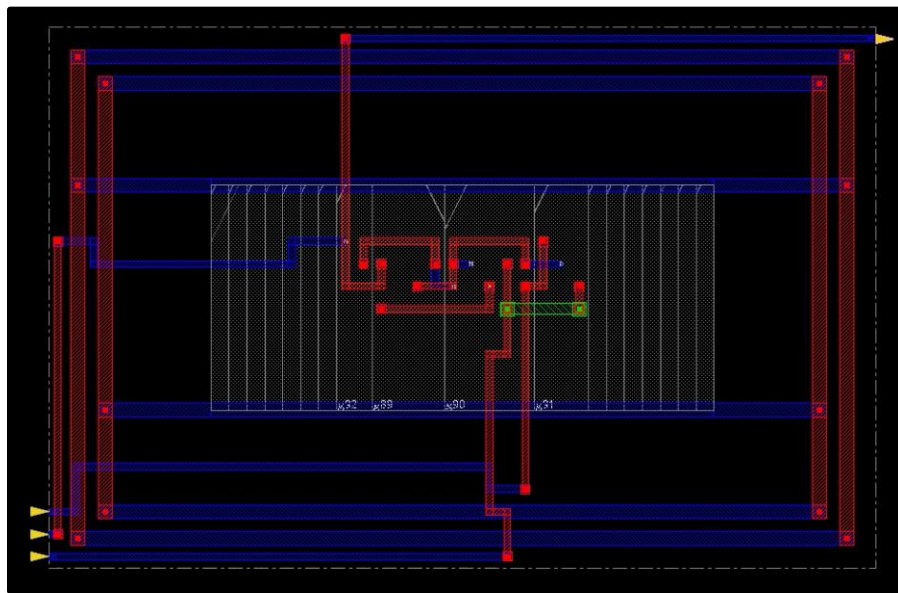


If you see white boxes and spacing errors when you route, your design is too constrained. Consider increasing the offset of your power rings by increments of 0.15.

Filler Cells

If you import the complete layout from Innovus into Cadence at this point, you may have DRC errors due to nwell layers not being spaced far enough. You can fix these errors by hand in Cadence or you can add pre-designed filler cells with Innovus. These cells provide continuity for nwell layers and power rails between placed cells.

Filler cells are placed by selecting the *Place* menu and then *Physical Cell* → *Add Filler*. In the dialog box that appears, click “Select” in the Cell Name(s) field and select the appropriate fill layers. **FILLER CELLS IN THE OSU05 LIBRARY ARE JUST CALLED ‘FILL’ AND THERE IS ONLY ONE OF THEM.** You can now click OK to fill in the blank regions in the core of your layout.



Verification

Before exporting your layout, you need to verify that there are not any geometric errors that occurred during placement or connectivity errors from routing.

This can be done by selecting *Verify Geometry* and *Verify Connectivity* in the *Verify* menu. Reports will be generated and placed in the directory that you started Innovus from. You can also look at the terminal used to start Innovus and see the results of the verification. If errors occurred, they are

likely the result of your area being too compact. If this is the case, consider increasing your power ring offsets by an increment of 0.15 and/or increasing your Core Margins slightly.

Submit valid pictures of the Innovus Connectivity and Geometry reports from the Terminal as a checkpoint.

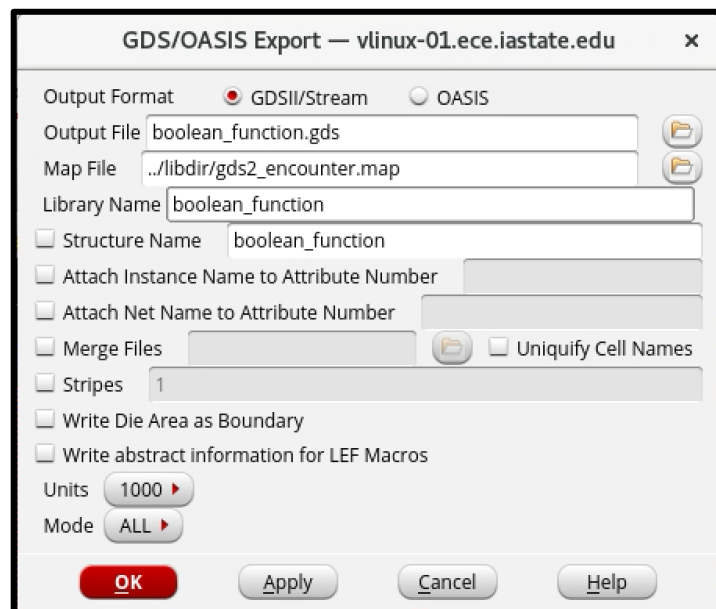
GDS Export of Layout from Innovus

Once the layout is complete in Innovus (post-routing and post-filler cells), then you need to perform an export of the layout. This is done in Innovus by going to *File* → *Save* → *GDS/OASIS..*

In the window that appears, select the *GDSII* radio button. Then type an output file name in 'Output File' field such as *boolean_function.gds*. In the map field select the '*gds2_encounter.map*' file located in the *libdir* directory.

In the Library Name, type the Library in Cadence you would like to add the design to. Use the same library you used to import the Verilog schematic.

Leave all other fields as default making sure the units are '1000' are mode is 'ALL'. Click OK. Look at the terminal and make sure Stream out has finished successfully.



Part 3: Import the layout into Cadence

The generalized digital circuit design flow, with the topics discussed in this tutorial highlighted, can be found below:

1. Design in HDL (Verilog file)
2. RTL Compiler (Verilog file → Synthesized Verilog File)
3. Innovus (Synthesized Verilog file → Layout)
4. **Cadence**
 - a. Layout Import (Innovus → CIW Import Stream)
 - b. Netlist Import (Synthesized Verilog File → Verilog Import)
5. LVS Verification

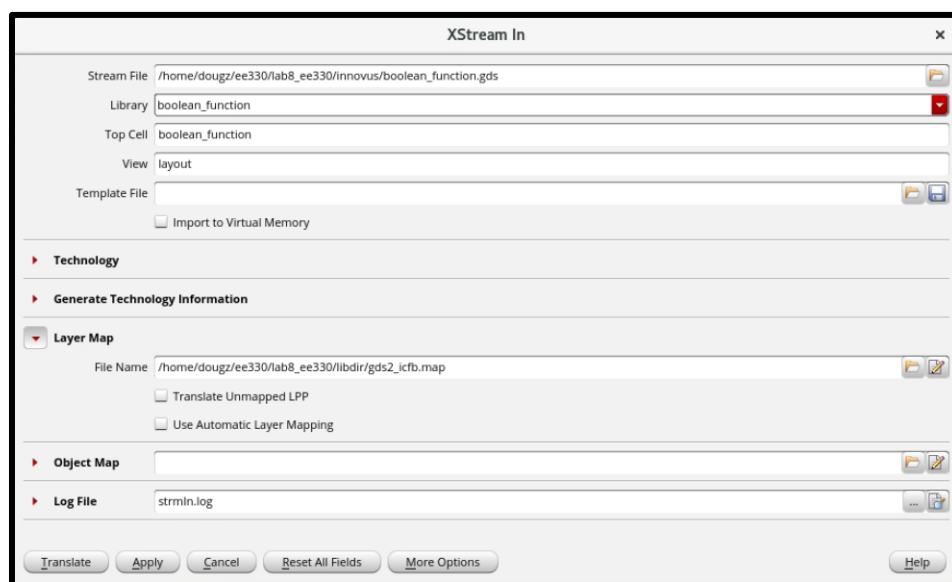
GDS Import of Layout into Virtuoso

After creating a layout in Innovus and creating the GDS file, open Virtuoso. Click on the CIW (Virtuoso window where errors and messages appear) and click *File → Import → Stream*. This may produce an error the first time. Try again and it should work and bring up an import window.

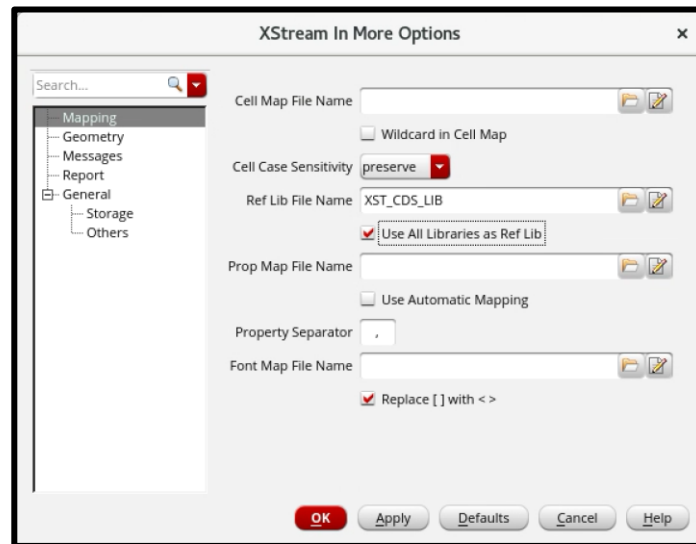
On the “stream in” window choose the GDS file created in Innovus as the ‘Stream In’ file.

Under library, type the name of the library you would like to save the module in. Note, this does need to be the same as the name of the library given in the stream out in Innovus. Then give a name to the top level cell once it is imported.

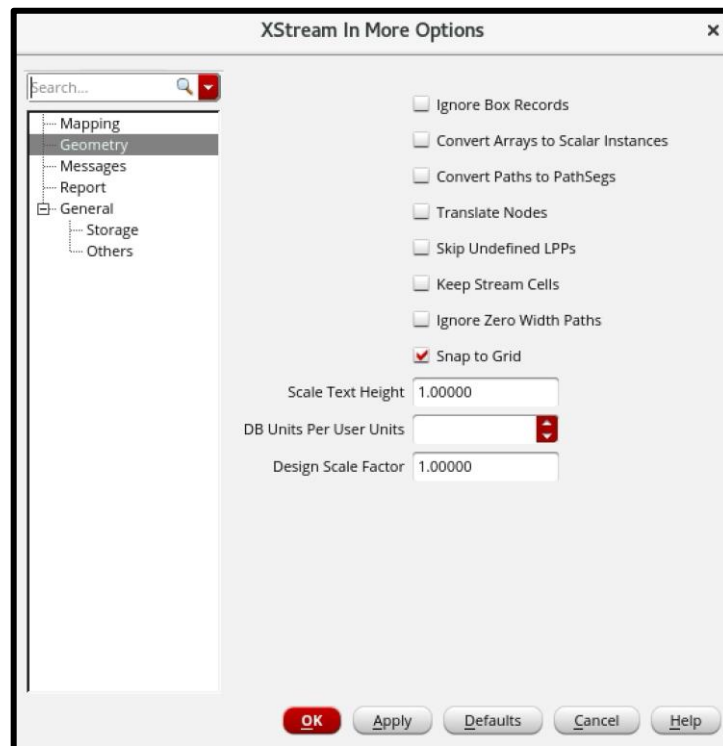
Under “Layer Map”, select the *gds2_icfb.map* file instead of the defaulted *NCSU_TechLib_ami06.layermap*.



Do not click Translate! First, go to *More Options*, then in the *Mapping* tab and check the “Replace [] with <>” box and “Use All Libraries as Ref Lib” box.



Now, go to the Geometry tab. Select Snap To Grid.



Now we are ready to translate the file and import the design. Click *Translate*. If a box to save the libraries comes up make sure to save the libraries as something. The process will take a little bit. When complete there should be a log file and a message box that appears. This should be 0 errors and only a few warnings, if not 0.

Go to the library created in the Library Manager and try to open the layout of the design. The standard cells should appear. Hit *Ctrl+F* to see the layout view.

Possible Errors

Red Cells with no Layers

If the cells appear but are red and disappear when you press Shift+F then there is a problem with the technology file in the folder. You should try copying the layout into a library with a known good technology file. If that does not work delete the library and attempt to stream it in again.

This can be solved through a slightly complicated process.

1. Copy the current library. This is just good practice.
2. Delete the original library.
3. Create the library from in Cadence (with the proper name, normally boolean_function)
4. Import the schematic created from RC5.Stream in the file from Innovus, but do not check (or uncheck) the box “Use all libraries as ref. Lib” in the More Options -> Mapping area.
5. Stream in the file again, this time with the “Use all libraries as ref. Lib” checked. This should give 0 errors and 7 warnings
6. Check the Layout and see if it worked.

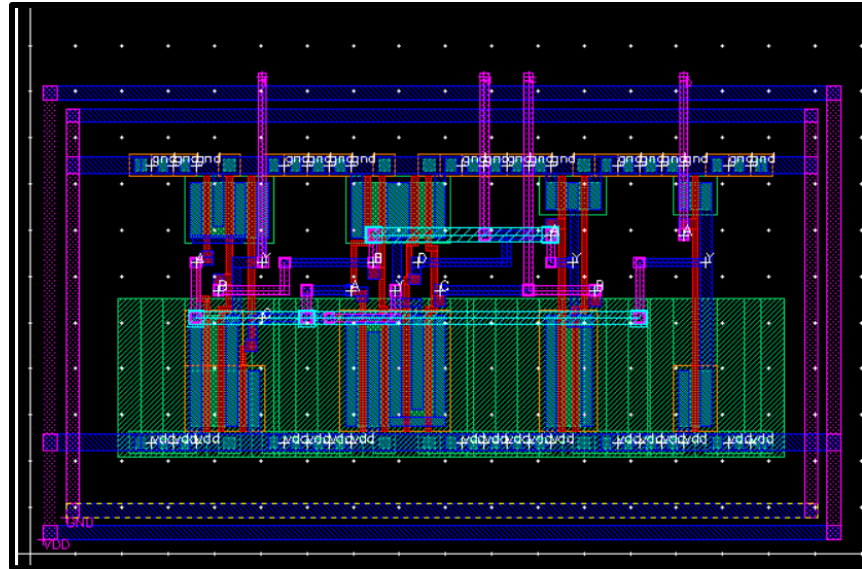
DRC and LVS error corrections

At this point, the synthesized schematic and layout from Innovus have been imported into Virtuoso. **Run a DRC and LVS check.** Both will fail, but it is critical that you run the checks before continuing.

The DRC will not pass due to “dubiousData” errors, the Innovus stream is creating metal layer text that should have been text layer. These can be deleted, but before doing so you should add pins.

Innovus is adding the name of what the pin should be where the pin should be placed, but does not actually create the pins. We advise creating the pins with the label name then deleting the label.

VDD here is the global VDD which is named “vdd!” and ground is the global ground which is named “gnd!”



Submit a valid DRC and LVS for this layout as a checkpoint.

Congratulations, you have synthesized your digital circuit! In your lab report, please provide a summary of the steps required for synthesizing and performing layout with Cadence RTL Compiler and SoC Innovus. Explain what the main steps do and provide screen shots of major steps as well as the two verification steps at the end.